

Fundamentos de Bases de Datos

Práctica- 3



Rodrigo Díaz-Regañón Ureña
Daniel Martínez Fernández

1. Introducción y objetivos

Esta práctica trata sobre la implementación de un gestor de bases de datos simplificado escrito en lenguaje C. El objetivo principal es gestionar el almacenamiento persistente de datos mediante registros binarios, sin depender de un gestor SQL externo.

En concreto, el proyecto modela la base de datos de una biblioteca, concentrándose en la información relacionada con los libros (identificador, ISBN, título y editorial). Para optimizar el rendimiento, se mantiene un índice en memoria que permite el acceso rápido a los registros almacenados en disco.

El desarrollo abarca la implementación de funciones clave como la búsqueda, el borrado y la inserción de registros. Un aspecto fundamental de esta práctica es la gestión del espacio en disco; para ello, se administran los huecos dejados por los registros borrados utilizando diversas estrategias de asignación de espacio: *Best Fit*, *First Fit* y *Worst Fit*.

Finalmente, se asegura la persistencia del sistema guardando tanto el índice como la lista de registros borrados en ficheros auxiliares al finalizar la ejecución, y se utiliza valgrind para verificar la correcta gestión de la memoria dinámica y la ausencia de fugas.

2. Estructura de los ficheros de datos e índices

Para el desarrollo de la práctica, el sistema maneja tres tipos de estructuras binarias:

1. Fichero de datos (.db): Almacena los registros de longitud variable. Cada registro comienza con un entero *size_t* (tamaño total), seguido del *bookID*, el *ISBN* y las cadenas variables (*Title*, *Editorial*) separadas por el delimitador |.
2. Fichero de índice (.ind): Contiene la memoria del array de índices. Cada elemento es una estructura fija con tres campos: *key* (*ID*), *offset* (posición en .db) y *size*. Esto permite cargar rápidamente el índice al iniciar el programa.
3. Fichero de huecos (.lst): Almacena la lista de registros borrados. El fichero comienza con un entero que indica la estrategia utilizada (*worst_fit*, *best_fit*, *first_fit*) seguido de estructuras que contienen el *offset* y el tamaño del hueco libre.

3. Implementación

3.1. Estructura del bucle principal

El núcleo del programa reside en la función `main`, donde se ha implementado un bucle infinito `while(1)` encargado de gestionar la interacción con el usuario mediante la línea de comandos. Para la lectura de comandos se utiliza `fgets` para leer el prompt, almacenando la línea completa en un buffer auxiliar. Es necesario procesar esta entrada eliminando el salto de línea residual (`\n`) mediante `strcspn` para evitar errores.

El análisis del comando introducido se realiza mediante comparaciones de cadenas con `strncmp` y `strcmp`. Esto permite identificar qué acción desea realizar el usuario (*add*, *del*, *find*, *printInd*, etc.). Si se detecta el comando `exit`, el bucle se rompe (`break`), permitiendo que el programa proceda a las rutinas de guardado y liberación de memoria antes de finalizar.

```
while (1)
{
    printf("Type command and argument/s. Type exit to stop\n");
    printf("> ");
    fflush(stdout);

    if (!fgets(aux, LIB_SIZE, stdin))
        break;

    /* eliminar salto de línea */
    aux[strcspn(aux, "\n")] = 0;

    /* comando exit */
    if (strcmp(aux, "exit") == 0)
    {
        printf("exit\n");
        break;
    }

    /* comando add */
    if (strncmp(aux, "add ", 4) == 0)
    {
        args = aux + 4;

        /* dividir por | y empiezo por bookID */
        token = strtok(args, "|");
        if (!token)
            continue;
        book_aux.bookID = atoi(token);

        posicion = 0;

        if (posicion < (int)indices.used && indices.array[posicion].key == book_aux.bookID)
        {
            printf("Record with BookID=%i exists\n", book_aux.bookID);
            continue;
        }

        /*Ahora el isbn*/
        token = strtok(NULL, "|");
        strncpy(book_aux.isbn, token, ISBNSIZE);
    }
}
```

En esta imagen vemos el comienzo del bucle, como se leen los comandos y el comando *exit* para salir del bucle principal.

Para los comandos que requieren argumentos (como *add* o *del*), se ha utilizado la función *strtok* para segmentar la cadena de entrada.

3.2. Implementación del comando ADD

El primer comando implementado ha sido *add*. El proceso de inserción de elementos sigue los siguientes pasos:

1. Tras identificar el comando *add*, se extraen los tokens. Dado que el Título y la Editorial son de longitud variable, se reserva memoria dinámica (*malloc*) ajustada exactamente al tamaño de la cadena leída más el terminador o separador correspondiente.

```
/*Ahora el isbn*/
token = strtok(NULL, "|");
strncpy(book_aux.isbn, token, ISBNSIZE);

/*Ahora el titulo con el separador */
token = strtok(NULL, "|");
if (!(book_aux.title = malloc(strlen(token) + 2)))
{
    fprintf(stderr, "Error reservando memoria para title\n");
    return -1;
}
sprintf(book_aux.title, "%s|", token);

/*Por ultimo el autor*/
token = strtok(NULL, "|");
if (!(book_aux.printedBy = malloc(strlen(token) + 1)))
{
    fprintf(stderr, "Error reservando memoria para title\n");
    return -1;
}
sprintf(book_aux.printedBy, "%s", token);
```

2. Antes de proceder a la escritura, el programa verifica que el bookID no exista ya en la base de datos. Para ello, se consulta el array de índices en memoria (`indices.array`). Si la clave ya existe, se aborta la operación y se notifica al usuario.

```
while(posicion < (int)indices.used){
    if (indices.array[posicion].key == book_aux.bookID)
    {
        printf("Record with BookID=%i exists\n", book_aux.bookID);
        break;
    }
    posicion++;
}

if(posicion != (int)indices.used){
    continue;
}
```

3. Se calcula el tamaño total en bytes que ocupará el registro en disco. Este cálculo incluye el tamaño de los tipos de datos fijos (*int*, *char* array del ISBN) y la longitud de las cadenas dinámicas.
4. Dependiendo de la estrategia seleccionada al iniciar el programa (*best_fit*, *first_fit*, *worst_fit*), se invoca a la función correspondiente (*add_best_fit*, etc.). Estas funciones buscan en la lista de registros borrados (*Array_del*) un hueco con tamaño suficiente. Si se encuentra un hueco válido, la función devuelve su *offset* y gestiona la fragmentación sobrante. Si no hay huecos válidos o la lista está vacía, la función devuelve el *offset* del final del archivo (*fseek* con *SEEK_END*), haciendo crecer el fichero .db.
5. Por último, con el offset determinado, se posiciona el puntero del archivo y se escriben los datos mediante *fwrite*. Inmediatamente después, se crea una estructura *indexbook* con los datos del nuevo libro y se inserta en el array de índices (*insertArray_add*), manteniendo el índice en memoria sincronizado con el disco en todo momento.

```

/*Calculamos lo que va a ocupar*/
l1 = strlen(book_aux.title);
l2 = strlen(book_aux.printedBy);
size = sizeof(int) + sizeof(char) * ISBN_SIZE + sizeof(char) * l1 + sizeof(char) * l2;

/* escribir entrada la lista de indices */

if (strat == FIRSTFIT)
{
    offset = add_first_fit(size, &array_del, db);
}
else if (strat == BESTFIT)
{
    offset = add_best_fit(size, &array_del, db);
}
else
{
    offset = add_worst_fit(size, &array_del, db);
}

if (offset == -1)
{
    fprintf(stderr, "Error añadiendo un nuevo dato\n");
    return -1;
}
fseek(db, offset, SEEK_SET);

/* escribir registro en .db */
fwrite(&size, sizeof(size_t), 1, db);           /*Primero lo que ocupa*/
fwrite(&book_aux.bookID, sizeof(int), 1, db);    /*Ahora escribimos el ID*/
fwrite(book_aux.isbn, sizeof(char), ISBN_SIZE, db); /*El isbn*/
fwrite(book_aux.title, sizeof(char), l1, db);    /*El titulo*/
fwrite(book_aux.printedBy, sizeof(char), l2, db); /*El autor*/

idx.key = book_aux.bookID;
idx.offset = offset;
idx.size = size;

insertArray_add(&indices, idx);

printf("Record with BookID=%d has been added to the database\n", book_aux.bookID);

free(book_aux.title);
free(book_aux.printedBy);

```

En esta foto se ve desde el paso 3 hasta el final de la implementación del *add*.

3.3. Implementación del comando FIND

La recuperación de datos se realiza mediante el comando *find*. Para ello aprovechamos el índice cargado en memoria RAM para realizar un acceso directo en vez de una búsqueda secuencial en el disco. La lógica implementada sigue los siguientes pasos:

1. Se utiliza el algoritmo de búsqueda binaria sobre el array de estructuras *indexbook*. Al estar este array siempre ordenado por clave primaria (*key*), la complejidad de la búsqueda se reduce a $O(\log n)$. La función devuelve la posición en el array del índice donde se encuentra el libro.

```
int bin_search(Array_add *a, int key)
{
    int low = 0;
    int high = (int)a->used - 1;

    while (low <= high)
    {
        int mid = low + (high - low) / 2;

        if (a->array[mid].key == key)
            return mid;

        if (a->array[mid].key < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}
```

2. Si el índice devuelve una posición válida, obtenemos el *offset* y el *size*. Con estos datos, posicionamos el puntero de lectura del fichero *.db* exactamente al inicio del registro usando *fseek(db, offset, SEEK_SET)*.
3. Una vez posicionado, se lee el *bookID* y el *ISBN*. Para los campos de longitud variable (*Title* y *Editorial*), leemos el resto del registro en un buffer y utilizamos *strtok* con el delimitador `|` para separar las cadenas y mostrarlas por pantalla con el formato *ID|ISBN|Titulo|Editorial*.


```

/*comando find buscar elemento en el archivo de datos binario y sacar todos sus elementos*/
if (strncmp(aux, "find ", 5) == 0)
{
    args = aux + 5;

    token = strtok(args, "\n");
    if (!token)
        continue;
    aux_id = atoi(token);

    posicion = bin_search(&indices, aux_id);
    if (posicion == -1)
    {
        printf("Record with bookId=%i does not exist\n", aux_id);
        continue;
    }
    aux_ind = indices.array[posicion];
    /*castameos a lo que queremos que se convierta el tipo de dato : ej 3A 30 00 00 lo casteamos en int y sacamos el entero 12345*/
    fseek(db, aux_ind.offset + (int)sizeof(size_t), SEEK_SET);
    fread(&book_aux.bookID, (int)sizeof(int), 1, db);
    fread(&book_aux.isbn, ISBN_SIZE, 1, db);
    book_aux.isbn[ISBN_SIZE] = '\0';
    /*como es tamaño variable cogemos el resto del registro y luego separamos gracias a '|' */
    fread(buffer, aux_ind.size - (int)sizeof(int) - ISBN_SIZE, 1, db);
    buffer[aux_ind.size - (int)sizeof(int) - ISBN_SIZE] = '\0';
    token = strtok(buffer, "|");
    strcpy(title, token);
    title[sizeof(title)-1] = '\0';

    token = strtok(NULL, "");
    strcpy(printedBy, token);
    printedBy[sizeof(printedBy)-1] = '\0';

    printf("%i|%s|%s\n", book_aux.bookID, book_aux.isbn, title, printedBy);
    continue;
}

```

3.4. Implementación del comando DEL

El comando *del* no borra los datos físicamente del fichero .db ya que es costoso el movimiento de bytes en disco. En su lugar, se marca el espacio como "disponible" para futuras escrituras. El comando *del* sigue la siguiente lógica:

1. Se comprueba mediante *bin_search* que el libro a borrar existe en el índice.
2. Se elimina la entrada correspondiente del array *indices*. Para ello, se utiliza la función *deleteArray_add*, la cual usa *memmove* para desplazar los elementos posteriores una posición hacia la izquierda, compactando el array y evitando huecos en la estructura en memoria.
3. Por último, se toman el *offset* y el *size* del libro eliminado y se insertan en la lista de huecos libres. Aquí es donde se escoge entre las distintas estrategias de gestión de momia (*Best*, *First o Worst Fit*). Dependiendo de la estrategia con la que se inició el programa, se llama a la función de inserción específica (*insertArray_del_ff*, *insertArray_del_bf* o *insertArray_del_wf*). Estas funciones insertan el nuevo hueco en la lista *Array_del* manteniendo un orden específico para facilitar la búsqueda de huecos en futuras operaciones *add*.

```

/*Delete*/
if (strncmp(aux, "del ", 4) == 0)
{
    args = aux + 4;

    token = strtok(args, "\n");
    if (!token)
        continue;
    aux_id = atoi(token);

    /*comprobamos que existe el dato en array de indices para asi poder borrarlo*/
    posicion = bin_search(&indices, aux_id);
    if (posicion == -1)
    {
        printf("Item with key %i does not exist\n", aux_id);
    }

    /*eliminamos el elemento de array de indices*/
    aux_ind = indices.array[posicion];
    deleteArray_add(&indices, posicion);

    /*Conseguimos los datos del elemento indexdeletedbook que queremos anadir al array de borrados */
    aux_deleted.offset = aux_ind.offset;
    aux_deleted.register_size = aux_ind.size;
    if (strat == FIRSTFIT)
    {
        insertArray_del_ff(&array_del, aux_deleted);
    }
    else if (strat == BESTFIT)
    {
        insertArray_del_bf(&array_del, aux_deleted);
    }
    else
    {
        insertArray_del_wf(&array_del, aux_deleted);
    }
    printf("Record with BookID=%i has been deleted\n", aux_ind.key);
    continue;
}

```

En concreto, las funciones de inserción funcionan de la siguiente manera:

- *insertArray_del_bf* (Best Fit): Mantiene la lista ordenada por tamaño de menor a mayor. Esto permite que, al buscar un hueco, el primero que quepa sea el que menos espacio desperdicie (el "mejor ajuste").
- *insertArray_del_wf* (Worst Fit): Mantiene la lista ordenada por tamaño de mayor a menor. Esto coloca el hueco más grande siempre en la primera posición (índice 0), facilitando encontrar el hueco donde sobra más espacio.
- *insertArray_del_ff* (First Fit): Inserta manteniendo el orden de mayor a menor. Su función es mantener la lista organizada para que la función *add_first_fit* pueda recorrerla y detenerse en el primer hueco válido que encuentre.

Por ejemplo, *insertArray_del_ff*:

```

void insertArray_del_ff(Array_del *a, indexdeletedbook element)
{
    /* insert item "element" in array
    a->used is the number of used entries,
    a->size is the number of entries */
    size_t pos = 0;

    if (a->used == a->size)
    {
        a->size *= 2;
        a->array = realloc(a->array, a->size * sizeof(indexdeletedbook));
    }
    while (pos < a->used && a->array[pos].register_size >= element.register_size)
        pos++;

    /* Hago hueco con memmove*/
    memmove(&a->array[pos + 1], &a->array[pos], (a->used - pos) * sizeof(indexdeletedbook));

    /*inserto */
    a->array[pos] = element;
    a->used++;
}

```

3.5. Implementación del comando PRINT

Para verificar el estado interno de las estructuras de datos, se han implementado tres comandos:

1. *printInd*: Recorre el array *indices* e imprime por pantalla la *key*, *offset* y *size* de cada libro. Sirve para comprobar que el índice se mantiene ordenado tras inserciones y borrados.
2. *printLst*: Muestra el contenido de la lista de huecos (*Array_del*). Sirve para comprobar si las estrategias de reutilización de espacio están funcionando y si los huecos se están registrando.
3. *printRec*: Recorre el índice y, para cada entrada, accede al disco (*fseek* + *fread*) para mostrar la información completa del libro (incluyendo título y editorial). A diferencia de leer el fichero *.db* secuencialmente, este comando solo muestra los libros activos, saltándose los registros borrados físicamente.

```
/* comando printInd */
if (strcmp(aux, "printInd") == 0)
{
    for (i = 0; i < (int)indices.used; i++)
    {
        printf("Entry #%d\n", i);
        printf("  key: %d\n", indices.array[i].key);
        printf("  offset: %ld\n", indices.array[i].offset);
        printf("  size: %ld\n", indices.array[i].size);
    }
    continue;
}

/* comando printLst */
if (strcmp(aux, "printLst") == 0)
{
    for (i = 0; i < (int)array_del.used; i++)
    {
        printf("Entry #%d\n", i);
        printf("  offset: %ld\n", array_del.array[i].offset);
        printf("  size: %ld\n", array_del.array[i].register_size);
    }
    continue;
}

/* comando printRec*/
if (strcmp(aux, "printRec") == 0)
{
    for (i = 0; i < (int)indices.used; i++)
    {
        aux_ind = indices.array[i];
        /*casteamos a lo que queremos que se convierta el tipo de dato : ej 3A 30 00 00 lo casteamos en int y sacamos el entero 12345*/
        fseek(db, aux_ind.offset + (int)sizeof(size_t), SEEK_SET);
        fread(&book_aux.bookID, (int)sizeof(int), 1, db);
        fread(&book_aux.isbn, ISBN_SIZE, 1, db);
        book_aux.isbn[ISBN_SIZE] = '\0';
        /*como es tamaño variable cogemos el resto del registro y luego separamos gracias a '|' */
        fread(buffer, aux_ind.size - (int)sizeof(int) - ISBN_SIZE, 1, db);
        buffer[aux_ind.size - (int)sizeof(int) - ISBN_SIZE] = '\0';
        token = strtok(buffer, "|");
        strcpy(title, token);
        title[sizeof(title)-1] = '\0';
        token = strtok(NULL, "\0");
        strcpy(printedBy, token);
        printedBy[sizeof(printedBy)-1] = '\0';

        printf("%i|%s|%s|%s\n", book_aux.bookID, book_aux.isbn, title, printedBy);
    }
    continue;
}
```

3.6. Carga y cierre del programa

Previo a la ejecución del bucle de comandos, se intenta la apertura y lectura de los archivos *.ind* y *.lst* para restaurar el estado anterior. Los datos se vuelcan en memoria dinámica utilizando *load_ind_to_array_add* para el índice y *load_ind_to_array_del* para los registros borrados. Es importante destacar que el fichero *.lst* incluye en su inicio un entero que indica la estrategia de ajuste usada (0=Best Fit, 1=Worst Fit, 2=First Fit). El programa valida este valor frente a la estrategia actual y, si no coinciden, reconstruye el orden de la lista en memoria para adaptarla a la nueva configuración.

```
db = fopen(datos, "r+b");
if (db == NULL)
{
    /* Si falla es porque no existe, lo creamos con w+b (Leer/Escribir) */
    db = fopen(datos, "w+b");
    if (!db)
    {
        perror("Error al crear el fichero de datos\n");
        return 0;
    }
}
ind = fopen(indice, "r+b");
if (ind == NULL)
{
    ind = fopen(indice, "w+b");
    if (!ind)
    {
        perror("Error al crear el fichero de índices\n");
        return 0;
    }
}

lst = fopen(lista, "r+b");
if (!lst)
{
    lst = fopen(lista, "w+b");
    if (lst == NULL)
    {
        perror("Error al crear el fichero de datos\n");
        return 0;
    }
}

/*inicializo array para indices*/
initArray_add(&indices, 2);
if (indices.array == NULL)
{
    perror("Error al crear el array de índices\n");
    return 0;
}

load_ind_to_array_add(&indices, ind);

/* inicializo array para borrados*/
initArray_del(&array_del, 2);
if (array_del.array == NULL)
{
    perror("Error al crear el array de borrados\n");
    return 0;
}

load_ind_to_array_del(&array_del, lst, strat);
```

Al escribir el comando `exit`, el programa vuelca el contenido de los arrays *indices* y *array_del* a sus respectivos ficheros binarios (*fwrite*). Finalmente, se libera toda la memoria reservada con *freeArray_add* y *freeArray_del* y se cierran los ficheros (*fclose*), asegurando una salida limpia sin fugas de memoria.

```
fseek(ind, 0, SEEK_SET);
for (i = 0; i < (int)indices.used; i++)
{
    fwrite(&indices.array[i].key, sizeof(int), 1, ind);
    fwrite(&indices.array[i].offset, sizeof(long int), 1, ind);
    fwrite(&indices.array[i].size, sizeof(size_t), 1, ind);
}

fseek(lst, 0, SEEK_SET);
fwrite(&strat, sizeof(int), 1, lst);
for (i = 0; i < (int)array_del.used; i++)
{
    fwrite(&array_del.array[i].offset, sizeof(size_t), 1, lst);
    fwrite(&array_del.array[i].register_size, sizeof(size_t), 1, lst);
}

fclose(db);
fclose(ind);
fclose(lst);
freeArray_add(&indices);
freeArray_del(&array_del);

printf("all done\n");
return 0;
```

```
==25870==
==25870== HEAP SUMMARY:
==25870==    in use at exit: 0 bytes in 0 blocks
==25870==   total heap usage: 15 allocs, 15 frees, 17,266 bytes allocated
==25870==
==25870== All heap blocks were freed -- no leaks are possible
==25870==
==25870== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Se puede observar que el `valgrind` no da ningún error. Esto es así para cualquier secuencia de comandos ejecutables.

4. Pruebas y resultados

Para verificar el correcto funcionamiento de la implementación, se han realizado diversas pruebas utilizando tanto los scripts proporcionados en el enunciado (*add_data_test.sh*, *add_delete_test_01.sh*, etc.) como nuevos scripts para verificar la gestión de huecos y el *printRec* y el *find*.

Todos los test proporcionados en el enunciado funcionan porque en todos se devuelve el mismo output con todas las partes funcionando correctamente. Además para aquellos que usan valgrind se ha revisado que no surgiese ningún error durante su ejecución y que se liberase toda la memoria reservada adecuadamente.

```
all done
1) Delete index records plus list of deleted books OK, ;-)
2) file test.ind Exists, ;-)
3) delete books files are identical, ;-)
4) Script end
```

Los nuevos scripts son los siguiente:

- *test_use_deleted_books_ff*, *test_use_deleted_books_wf* y *test_use_deleted_books_bf* son tests diseñados para verificar la reutilización de espacio según el tipo de estrategia utilizada.
- *test_find_printRec* es un test que inserta un gran número de datos y comprueba el correcto funcionamiento del comando *find* y su posterior impresión con el comando *printRec*.

La prueba de estos tests se realizó con valgrind, y se pudo observar como no aparece ningún error durante sus respectivas ejecuciones:

Para mostrar que funciona, adjuntamos la ejecución con pantallazos que dió el test, *test_use_deleted_books_ff*, estas capturas serían similares para el resto de estrategias:

1.

```
spawn rm -f test_reuse.db test_reuse.ind test_reuse.lst
spawn valgrind --track-origins=yes ./library first_fit test_reuse
==3408== Memcheck, a memory error detector
==3408== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==3408== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==3408== Command: ./library first_fit test_reuse
==3408==
Estrategia seleccionada: first_fit
Archivos a crear:
Datos: test_reuse.db
Indice: test_reuse.ind
Listado: test_reuse.lst
Type command and argument/s. Type exit to stop
add 12346|978-2-12345680-3|El Quijote|Catedra
> Record with BookID=12346 has been added to the database
Type command and argument/s. Type exit to stop
> add 12345|978-2-12345086-3|La busca|Catedra
Record with BookID=12345 has been added to the database
Type command and argument/s. Type exit to stop
> add 12347|978-2-12345680-4|Amar|catedra
Record with BookID=12347 has been added to the database
Type command and argument/s. Type exit to stop
> add 12348|978-2-12345086-3|la busca|catedra
Record with BookID=12348 has been added to the database
Type command and argument/s. Type exit to stop
> -----
printInd
Entry #0
  key: #12345
  offset: #46
  size: #36
Entry #1
  key: #12346
  offset: #0
  size: #38
Entry #2
  key: #12347
  offset: #90
  size: #32
Entry #3
  key: #12348
  offset: #130
  size: #36
Type command and argument/s. Type exit to stop
```

2.

```
> del 12346
Record with BookID=12346 has been deleted
Type command and argument/s. Type exit to stop
> printInd
Entry #0
  key: #12345
  offset: #46
  size: #36
Entry #1
  key: #12347
  offset: #90
  size: #32
Entry #2
  key: #12348
  offset: #130
  size: #36
Type command and argument/s. Type exit to stop
> del 12347
Record with BookID=12347 has been deleted
Type command and argument/s. Type exit to stop
> printInd
Entry #0
  key: #12345
  offset: #46
  size: #36
Entry #1
  key: #12348
  offset: #130
  size: #36
Type command and argument/s. Type exit to stop
> printLst
Entry #0
  offset: #0
  size: #38
Entry #1
  offset: #90
  size: #32
Type command and argument/s. Type exit to stop
> add 12347|978-2-12345680-1|La insoportable levedad del ser|Catedra
Record with BookID=12347 has been added to the database
Type command and argument/s. Type exit to stop
> add 12346|978-2-12345680-1|El Cuentacuentos maldito de los cien demon
Record with BookID=12346 has been added to the database
Type command and argument/s. Type exit to stop
```

3.

```
Entry #0
  key: #12345
  offset: #46
  size: #36
Entry #1
  key: #12346
  offset: #241
  size: #110
Entry #2
  key: #12347
  offset: #174
  size: #59
Entry #3
  key: #12348
  offset: #130
  size: #36
Type command and argument/s. Type exit to stop
> printLst
Entry #0
  offset: #0
  size: #38
Entry #1
  offset: #90
  size: #32
Type command and argument/s. Type exit to stop
> del 12345
Record with BookID=12345 has been deleted
Type command and argument/s. Type exit to stop
> printLst
Entry #0
  offset: #0
  size: #38
Entry #1
  offset: #46
  size: #36
Entry #2
  offset: #90
  size: #32
Type command and argument/s. Type exit to stop
> add 99999|978-2-00000000-0|Edito|Si
Record with BookID=99999 has been added to the database
Type command and argument/s. Type exit to stop
> printInd
Entry #0
```

4.

```
Entry #0
  key: #12346
  offset: #241
  size: #110
Entry #1
  key: #12347
  offset: #174
  size: #59
Entry #2
  key: #12348
  offset: #130
  size: #36
Entry #3
  key: #99999
  offset: #0
  size: #28
Type command and argument/s. Type exit to stop
> printLst
Entry #0
  offset: #46
  size: #36
Entry #1
  offset: #90
  size: #32
Entry #2
  offset: #28
  size: #10
Type command and argument/s. Type exit to stop
> exit
exit
all done
1) Logic Check: OK ;- )
2) file test_reuse.ind Exists, ;- )
3) index files are identical, ;- )
3) delete books files are identical, ;- )
4) Script end
```

5. Conclusión

Esta práctica nos ha servido para entender en profundidad cómo funciona internamente un gestor de datos y la importancia de optimizar recursos. Lo principal que hemos aprendido es la gran diferencia de rendimiento que existe entre acceder al disco y trabajar con memoria RAM; implementar el índice en memoria y la búsqueda binaria nos ha demostrado cómo se puede agilizar enormemente la localización de registros.

Uno de los mayores retos ha sido gestionar el borrado lógico. Al implementar el sistema de huecos y las estrategias de reutilización (como *Best Fit* o *Worst Fit*), hemos comprendido cómo se puede optimizar el espacio en disco sin el coste de mover datos físicamente cada vez que eliminamos algo.

Por último, trabajar con ficheros binarios y memoria dinámica nos ha obligado a ser mucho más rigurosos en la programación. El uso de herramientas como *Valgrind* ha sido fundamental para asegurarnos de que nuestro código es robusto y no deja fugas de memoria al terminar la ejecución.